

Phase 1: Tree Foundations and Binary Tree Basics

0. Purpose of This Phase

This phase teaches the basic language of trees in Data Structures and Algorithms.

Before learning tree formulas, traversals, recursion, strict trees, complete trees, or tree construction, you must first understand:

- What a tree is
- What nodes and edges are
- How parent-child relationships work
- What root, leaf, internal node, degree, level, and height mean
- What a binary tree is
- How trees are represented using pointers in C++
- What skewed binary trees are

The main beginner idea is:

A tree is made by connecting nodes using links or pointers.

1. What Is a Tree?

A **tree** is a non-linear data structure made of **nodes** and **edges**.

1.1 Node

A **node** is one data item in the tree.

Example:

A

Here, **A** is a node.

A node can store data such as:

- A character, like **A**
- A number, like **10**
- A string, like **"John"**

- An object or record

1.2 Edge

An **edge** is the connection between two nodes.

Example:



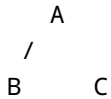
```
graph TD; A --- B;
```

Here:

- A is a node
- B is a node
- The line between A and B is an edge

The edge means A is connected to B.

1.3 Tree Example



```
graph TD; A --- B; A --- C;
```

In this tree:

- A, B, and C are nodes
- The line from A to B is an edge
- The line from A to C is an edge

So:

Tree = Nodes + Edges

2. Why Is a Tree Called Non-Linear?

A tree is called **non-linear** because data does not move in one straight line.

2.1 Linear Structure Example

An array is linear:

```
10 -> 20 -> 30 -> 40
```

Each item comes after another item in one direction.

2.2 Non-Linear Tree Example

A tree branches:

```
    10
   /
  20   30
```

Here, 10 connects to both 20 and 30.

So one node can lead to many other nodes.

That is why a tree is non-linear.

3. Root Node

The **root** is the topmost node of a tree.

Example:

```
    A
   /
  B   C
```

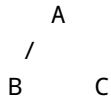
Here, A is the root.

Important rule:

```
The root has no parent.
```

Every other node in the tree has exactly one parent.

So in this tree:



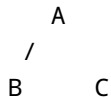
- A has no parent
- B has parent A
- C has parent A

4. Parent and Child

A **parent** is a node that directly connects downward to another node.

A **child** is a node directly below another node.

Example:



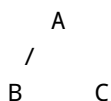
Here:

- A is the parent of B
- A is the parent of C
- B is a child of A
- C is a child of A

We can write it like this:

```
A -> parent
B, C -> children of A
```

4.1 Another Example



/
D

Here:

- A is parent of B and C
- B is parent of D
- D is child of B
- B is child of A
- C is child of A

Important:

A node can be both a child and a parent.

For example, B is:

- A child of A
- A parent of D

5. Siblings

Siblings are nodes that have the same parent.

Example:

A
/
B C

B and C are siblings because both have the same parent: A.

5.1 More Than Two Siblings

A
/ |
B C D

Here:

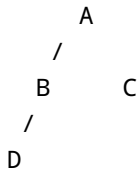
- B, C, and D are siblings

- They all have the same parent: **A**

6. Ancestors

An **ancestor** is any node above a given node on the path to the root.

Example:



For node **D**:

- **B** is above **D**
- **A** is above **B**

So the ancestors of **D** are:

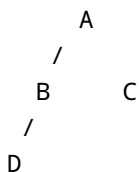
B, A

Simple meaning:

Ancestors are the nodes you meet when moving upward from a node to the root.

6.1 Ancestor Examples

Using this tree:



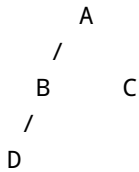
- Ancestors of **D** are **B** and **A**
- Ancestor of **B** is **A**
- Ancestor of **C** is **A**

- A has no ancestors because it is the root

7. Descendants

A **descendant** is any node below a given node.

Example:



The descendants of A are:

B, C, D

Because all of them are below A.

The descendants of B are:

D

Simple meaning:

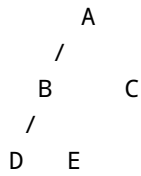
Descendants are all nodes you can reach by moving downward from a node.

8. Subtree

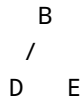
A **subtree** is a smaller tree inside a bigger tree.

It starts from a node and includes everything below that node.

Example:



The subtree rooted at **B** is:



This is a subtree because it contains:

- Node **B**
- All descendants of **B**

So:

Subtree = a node + all nodes under it

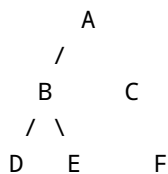
9. Forest

A **forest** is a collection of separate trees.

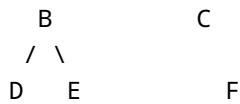
If you remove the root of a tree, the remaining child subtrees can become a forest.

Example:

Original tree:



If we remove root **A**, we get two separate trees:



These two separate trees together form a forest.

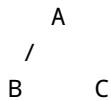
Simple meaning:

Forest = collection of trees

10. Leaf Node / External Node / Terminal Node

A **leaf node** is a node with no children.

Example:



Here:

- B has no children
- C has no children

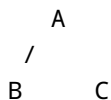
So B and C are leaf nodes.

Other names for leaf node are:

Leaf node = External node = Terminal node

All three names mean the same thing.

10.1 Leaf Example



/
D

Leaf nodes are:

C, D

Why?

- C has no children
- D has no children

A is not a leaf because it has children.

B is not a leaf because it has child D.

11. Internal Node / Non-Leaf Node

An **internal node** is a node that has at least one child.

Example:

A
/
B C

Here, A is an internal node because it has children.

Other name:

Internal node = Non-leaf node

11.1 Internal Node Example

A
/
B C
/
D

Internal nodes are:

A, B

Why?

- A has children B and C
- B has child D

Leaf nodes are:

C, D

Why?

- C has no children
- D has no children

12. Degree of a Node

The **degree of a node** means the number of direct children that node has.

Simple definition:

Degree of a node = number of direct children of that node

Example:

```
  A
 / |
B  C D
```

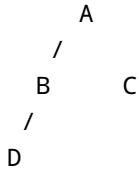
Node A has 3 children:

B, C, D

So:

$\text{degree}(A) = 3$

12.1 Degree Example



Degrees:

$\text{degree}(A) = 2$
 $\text{degree}(B) = 1$
 $\text{degree}(C) = 0$
 $\text{degree}(D) = 0$

Explanation:

- A has two children: B and C
- B has one child: D
- C has no children
- D has no children

Important:

A leaf node always has degree 0.

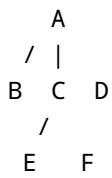
13. Degree of a Tree

The **degree of a tree** is the largest degree of any node in the tree.

Simple definition:

Degree of a tree = maximum degree among all nodes in the tree

Example:



Degrees:

```

degree(A) = 3
degree(B) = 0
degree(C) = 2
degree(D) = 0
degree(E) = 0
degree(F) = 0
  
```

The largest degree is **3**.

So:

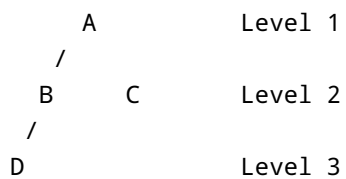
Degree of the tree = 3

14. Level

Level tells how far a node is from the top of the tree.

In this curriculum, the root starts at **Level 1**.

Example:



So:

```

Level of A = 1
Level of B = 2
  
```

Level of C = 2
Level of D = 3

Simple meaning:

Level counts nodes from the top downward.

Important:

Root is Level 1.

15. Height

Height counts edges from the root down to the deepest node.

In this curriculum, height starts at 0 for a single root node.

15.1 Single Node Tree

A

There are no edges.

So:

Height = 0

15.2 Tree with One Edge

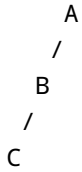
A
/
B

There is 1 edge from A to B.

So:

Height = 1

15.3 Tree with Two Edges



There are 2 edges from **A** to **C**.

So:

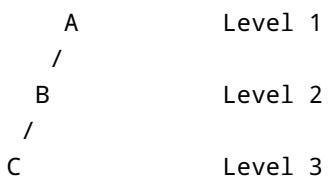
Height = 2

16. Level vs Height

This is one of the most important differences in Phase 1.

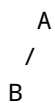
16.1 Level Counts Nodes

Level starts at 1.



16.2 Height Counts Edges

Height starts at 0.



/
C

The longest path is:

A -> B -> C

This path has 2 edges.

So:

Height = 2

16.3 Key Difference

Level counts nodes.
Height counts edges.

For a single root:

Level = 1
Height = 0

This difference matters later because many formulas use height starting from 0.

17. General Tree

A **general tree** is a tree where a node can have any number of children.

Example:

```
      A
     / |
    B  C D
     /
    E  F
```


Here:

- A has 3 children
- C has 2 children
- B, D, E, and F have 0 children

This is a general tree because nodes are not limited to only two children.

17.1 General Tree Node in C++

For a general tree, a node may have many children.

So we use a `vector` to store child addresses.

```
#include <iostream>
#include <vector>
using namespace std;

struct Node {
    char data;
    vector<Node*> children;

    Node(char val) {
        data = val;
    }
};

int main() {
    Node* root = new Node('A');
    Node* child1 = new Node('B');
    Node* child2 = new Node('C');

    root->children.push_back(child1);
    root->children.push_back(child2);

    cout << "Root: " << root->data << endl;
    cout << "Children: ";
    cout << root->children[0]->data << " ";
    cout << root->children[1]->data << endl;

    return 0;
}
```

17.2 Explanation of the Code

This line creates the root node:

```
Node* root = new Node('A');
```

This line creates a child node:

```
Node* child1 = new Node('B');
```

This line creates another child node:

```
Node* child2 = new Node('C');
```

This line connects **B** as a child of **A**:

```
root->children.push_back(child1);
```

This line connects **C** as a child of **A**:

```
root->children.push_back(child2);
```

The important idea is:

```
The vector stores addresses of child nodes.
```

So `root->children` does not store the full child nodes directly. It stores pointers to those child nodes.

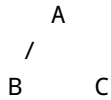
18. Binary Tree

A **binary tree** is a tree where each node can have at most two children.

The two children are called:

left child
right child

Example:



Here:

B is the left child of A.
C is the right child of A.

18.1 Possible Children in a Binary Tree

In a binary tree, a node can have:

0 children
1 child
2 children

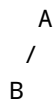
But it can never have more than 2 children.

18.2 Valid Binary Tree Examples

A node with 0 children:

A

A node with 1 left child:



A node with 1 right child:

A

B

A node with 2 children:

A

/

B

C

All of these are valid binary trees.

18.3 Invalid Binary Tree Example

A

/

B

|

C

D

This is not a binary tree because **A** has 3 children.

A binary tree allows at most 2 children per node.

19. Binary Tree Node in C++

Because a binary tree node has at most two children, we can store two pointers:

- one pointer for the left child
- one pointer for the right child

```
#include <iostream>
using namespace std;

struct BinaryNode {
    char data;
    BinaryNode* left;
    BinaryNode* right;

    BinaryNode(char val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};
```

```

    }
};

int main() {
    BinaryNode* root = new BinaryNode('A');

    root->left = new BinaryNode('B');
    root->right = new BinaryNode('C');

    cout << root->left->data << " <- ";
    cout << root->data;
    cout << " -> " << root->right->data << endl;

    return 0;
}

```

Expected output:

```
B <- A -> C
```

This output means:

```
A has left child B.
A has right child C.
```

20. Why Do We Use Pointers in Trees?

Trees are linked structures.

A node does not physically contain all of its children inside itself.

Instead, a node stores the addresses of its children.

For a binary tree node:

```
BinaryNode* left;
BinaryNode* right;
```

These are pointers.

Each pointer can either:

- point to another node
- store `nullptr` if no child exists

20.1 Example

```
root->left = new BinaryNode('B');
```

This means:

Create a new node B.
Store the address of B in root->left.

So this creates a connection from A to B.

Tree form:

```
  A
 /
B
```

20.2 nullptr

`nullptr` means the pointer is not pointing to any node.

When a new binary node is created, we usually set its child pointers to `nullptr`:

```
left = nullptr;
right = nullptr;
```

This means:

This node currently has no left child and no right child.

This is important because uninitialized pointers can contain garbage addresses.

21. Memory and Pointer Behavior

When we write:

```
BinaryNode* root = new BinaryNode('A');
```

The computer creates a node somewhere in memory.

`root` stores the address of that node.

When we write:

```
root->left = new BinaryNode('B');
```

The computer creates another node in memory and stores its address in `root->left`.

So the link between nodes is created using addresses.

Simple idea:

```
Pointers create the connections between tree nodes.
```

22. Skewed Binary Trees

A **skewed binary tree** is a binary tree that grows like a chain.

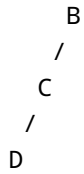
There are two types:

- Left-skewed binary tree
- Right-skewed binary tree

22.1 Left-Skewed Binary Tree

A **left-skewed binary tree** grows only to the left.

```
  A
 /
```

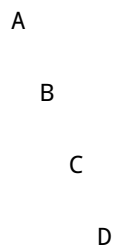


Every node has only a left child.

This is still a valid binary tree because no node has more than two children.

22.2 Right-Skewed Binary Tree

A **right-skewed binary tree** grows only to the right.



Every node has only a right child.

This is also a valid binary tree.

22.3 Why Skewed Trees Behave Like Linked Lists

A skewed tree has only one path downward.

Example:

A -> B -> C -> D

There is no real branching.

That is why skewed trees behave almost like linked lists.

23. Dry Run: Creating a Binary Tree

Consider this code:

```
BinaryNode* root = new BinaryNode('A');  
root->left = new BinaryNode('B');  
root->right = new BinaryNode('C');
```

Let us dry run it step by step.

Step 1

```
BinaryNode* root = new BinaryNode('A');
```

This creates node **A**.

A

root stores the address of **A**.

Step 2

```
root->left = new BinaryNode('B');
```

This creates node **B** and connects it as the left child of **A**.

```
  A  
 /  
B
```

Step 3

```
root->right = new BinaryNode('C');
```

This creates node **C** and connects it as the right child of **A**.

```
  A
 /
B  C
```

Final meaning:

```
B <- A -> C
```

So:

- **A** is the root
- **B** is the left child of **A**
- **C** is the right child of **A**
- **B** and **C** are siblings
- **B** and **C** are leaf nodes
- **A** is an internal node

24. Complete Phase 1 Example Program

This program shows both a general tree and a binary tree.

```
#include <iostream>
#include <vector>
using namespace std;

struct GeneralNode {
    char data;
    vector<GeneralNode*> children;

    GeneralNode(char val) {
        data = val;
    }
};

struct BinaryNode {
    char data;
    BinaryNode* left;
    BinaryNode* right;

    BinaryNode(char val) {
```

```

        data = val;
        left = nullptr;
        right = nullptr;
    }
};

int main() {
    GeneralNode* root = new GeneralNode('A');
    root->children.push_back(new GeneralNode('B'));
    root->children.push_back(new GeneralNode('C'));

    cout << "General Tree" << endl;
    cout << "Root: " << root->data << endl;
    cout << "Degree of root: " << root->children.size() << endl;
    cout << "Children: ";

    for (GeneralNode* child : root->children) {
        cout << child->data << " ";
    }

    cout << "\n\n";

    BinaryNode* bRoot = new BinaryNode('X');
    bRoot->left = new BinaryNode('Y');
    bRoot->right = new BinaryNode('Z');

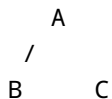
    cout << "Binary Tree" << endl;
    cout << bRoot->left->data << " <- " << bRoot->data << " -> ";
    cout << bRoot->right->data << endl;

    return 0;
}

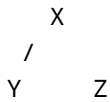
```

24.1 What This Program Does

First, it creates a general tree:



Then, it creates a binary tree:



24.2 Important Ideas from the Program

In the general tree:

```
vector<GeneralNode*> children;
```

This allows many children.

In the binary tree:

```
BinaryNode* left;  
BinaryNode* right;
```

This allows at most two children.

25. Time and Space Complexity for Basic Linking

When we create and connect a few nodes manually, each link is created in constant time.

Example:

```
root->left = new BinaryNode('B');
```

This is one direct operation.

So the time for one link is:

```
O(1)
```

The space used depends on how many nodes we create.

If we create `n` nodes, space is:

$O(n)$

Simple meaning:

More nodes require more memory.

26. Common Beginner Mistakes

Mistake 1: Confusing Level and Height

Wrong idea:

A single root has height 1.

Correct idea in this curriculum:

A single root has level 1.
A single root has height 0.

Why?

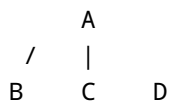
- Level counts nodes
- Height counts edges

A single root has no edge, so its height is 0.

Mistake 2: Thinking Every Tree Is Binary

Not every tree is binary.

This is a general tree:



A has 3 children.

So this is not a binary tree.

A binary tree allows at most 2 children per node.

Mistake 3: Storing Child Data Instead of Child Addresses

Beginners sometimes think we store the full child node inside the parent.

But in linked trees, we usually store pointers.

Correct idea:

```
root->left = new BinaryNode('B');
```

This stores the address of node `B` in `root->left`.

Mistake 4: Forgetting to Initialize Pointers to nullptr

Bad constructor:

```
BinaryNode(char val) {  
    data = val;  
}
```

Problem:

```
left and right are not initialized.
```

They may contain garbage addresses.

Better constructor:

```
BinaryNode(char val) {  
    data = val;  
    left = nullptr;  
    right = nullptr;  
}
```

This means the node starts with no children.

Mistake 5: Thinking a Leaf Cannot Be the Root

A single-node tree looks like this:

A

Here, **A** is both:

- the root
- a leaf

Why?

- It is the root because it is the top node
 - It is a leaf because it has no children
-

27. Phase 1 Key Vocabulary Table

Term	Simple Meaning
Tree	A non-linear structure made of nodes and edges
Node	A data item in the tree
Edge	A connection between two nodes
Root	The top node of a tree
Parent	A node directly above another node
Child	A node directly below another node
Siblings	Nodes with the same parent
Ancestors	Nodes above a node on the path to the root
Descendants	Nodes below a node
Subtree	A node together with all nodes under it
Forest	A collection of separate trees
Leaf / External / Terminal	A node with 0 children
Internal / Non-leaf	A node with at least 1 child
Degree of node	Number of direct children of that node

Term	Simple Meaning
Degree of tree	Largest node degree in the tree
Level	Position counted from the top, root is Level 1
Height	Longest edge path from root downward, root alone has height 0
General tree	A tree where a node can have any number of children
Binary tree	A tree where each node can have at most 2 children
Left child	The child connected on the left side
Right child	The child connected on the right side
nullptr	A pointer value meaning no node exists there
Left-skewed tree	A binary tree that grows only left
Right-skewed tree	A binary tree that grows only right

28. Phase 1 Summary

By the end of Phase 1, you should know the following:

```

Tree = nodes + edges
Root = top node
Parent = node above another node
Child = node below another node
Siblings = nodes with the same parent
Ancestors = nodes above a node toward the root
Descendants = nodes below a node
Subtree = a node and everything under it
Forest = collection of separate trees
Leaf = node with 0 children
Internal node = node with at least 1 child
Degree of node = number of direct children
Degree of tree = maximum degree among all nodes
Level = node-count position from top
Height = edge-count depth from root
General tree = node may have many children
Binary tree = each node has at most 2 children
Skewed tree = binary tree that grows like a chain

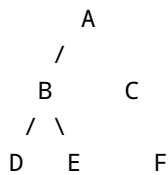
```

The most important memory idea:

A tree is built by connecting nodes using pointers.

29. Practice Questions

Use this tree:

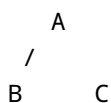


Answer these questions:

1. What is the root?
2. Who are the children of B?
3. Who is the parent of F?
4. Which nodes are siblings?
5. Which nodes are leaves?
6. Which nodes are internal nodes?
7. What is the degree of A?
8. What is the degree of B?
9. What is the degree of C?
10. What is the degree of the whole tree?
11. What is the level of A?
12. What is the level of D?
13. What is the height of the tree?
14. Is this a binary tree?
15. Is this a general tree?

30. Practice Answers

Using this tree:



/ \
D E F

1. What is the root?

A

2. Who are the children of B?

D and E

3. Who is the parent of F?

C

4. Which nodes are siblings?

B and C are siblings.
D and E are siblings.

5. Which nodes are leaves?

D, E, F

6. Which nodes are internal nodes?

A, B, C

7. What is the degree of A?

2

Because A has two children: B and C.

8. What is the degree of B?

2

Because B has two children: D and E.

9. What is the degree of C?

1

Because C has one child: F.

10. What is the degree of the whole tree?

2

The largest node degree is 2.

11. What is the level of A?

1

The root is Level 1.

12. What is the level of D?

3

Path:

A -> B -> D

So:

- A is Level 1
- B is Level 2
- D is Level 3

13. What is the height of the tree?

2

The longest root-to-leaf path has 2 edges.

Example:

A -> B -> D

Edges:

A to B = 1
B to D = 1
Total = 2

14. Is this a binary tree?

Yes

Why?

No node has more than 2 children.

15. Is this a general tree?

Yes

Why?

Every binary tree is also a kind of general tree, but not every general tree is binary.

31. Final Beginner Memory Trick

Remember this:

Tree vocabulary describes relationships.
Pointers create those relationships in memory.

So when you look at a tree, ask:

1. Which node is the root?
2. Which nodes are parents?
3. Which nodes are children?
4. Which nodes are leaves?
5. Which nodes are internal?
6. What is the degree of each node?
7. What is the level of each node?
8. What is the height of the tree?
9. Is it a general tree or a binary tree?
10. How would I connect it using pointers?

If you can answer these questions, you understand Phase 1 well.